Lecture 2 · Part I · Lecture notes

The end-to-end project

Dataset: California housing

How to use these notes

The primary text for this lecture is Géron, Chapter 2, with §4.1 brought forward. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 Where we are **2**

2 The split, before the exploration **2**

2.1 Data snooping is a bias, not a discipline problem . . . 3

2.2 Why random sampling is not automatically enough . . . 3

2.3 Does it actually matter? Measure it . . . 3

2.4 Assert after every structural step . . . 4

3 Exploring — the training set, and only that **4**

3.1 Look at it, and tune the plot until you can . . . 4

3.2 A default nobody chose, and how to catch it . . . 4

3.3 Correlations, and what they cannot see . . . 5

3.4 The cap, made visible — and the lines that are not there . . . 5

3.5 Attribute combinations . . . 6

4 Derivation: least squares, up to convexity **7**

4.1 The model . . . 7

4.2 What "fit" means . . . 7
4.3 The objective, its gradient, and its curvature . . . 7
4.4 Where the derivation stops . . . 8
4.4.1 Checked on our own data . . . 8

5 The notebook **8**

6 Design exercise: a rental marketplace's suggested listing price **9**

7 Exercises **11**

Summary **11**

1 Where we are

Lecture 1 framed the brief and looked at the whole dataset once. Five findings carry into today.

What we found	Why it matters today
The output is consumed as a *price*	supervised regression: one number per district, error in dollars
207 missing values in `total_bedrooms`; `ocean_proximity` is categorical, one level with $n = 5$	both must be handled before anything is fitted
A target capped: 965 districts at the cap, \$500,001	4.7% of districts carry a label that is not the answer
Scales four orders of magnitude apart, many columns skewed right	matters for some methods; not for the least squares derived today
Nothing split yet	so the split comes first, before we look any closer

Today has three parts: the split (§2), the exploration of the training set, and the first half of the mathematics of least squares (§4).

2 The split, before the exploration

Lecture 1 stopped four histograms in, before plotting one variable against another, because that is exactly the moment to stop. The only way to know how a model generalises is to try it on cases it has never seen: the data is split into a **training set** and a **test set**, and the error rate on new cases is the **generalisation error**. A low training error with a high generalisation error is **overfitting**.

2.1 Data snooping is a bias, not a discipline problem

Your brain is an outstanding pattern detector, which means it overfits. If you study the test set, you will — without any dishonesty, and without being able to point at the moment it happened — choose a model that suits it. Your estimate of generalisation error will be optimistic and you will ship something that underperforms.

The defence is procedural rather than moral: create the test set *before* exploring any further, and do not look at it again. Typically 80/20, though the proportion is a function of size — with ten million instances, holding out 1% is plenty.

2.2 Why random sampling is not automatically enough

Purely random sampling is fine if the dataset is large relative to the number of attributes. Ours may not be, and random sampling then risks **sampling bias**: a test set whose composition differs from the population, so that the error measured on it is an error on the wrong population.

Surveyors do not call 1,000 people at random; they ensure the sample is representative on the dimensions that matter. The experts told us median income predicts price, so the test set must be representative across income levels. That is **stratified sampling**: bin the stratifying variable, then sample within each bin in proportion.

The bins have to be chosen so that no stratum is too small to sample from. Five categories, cut at 1.5, 3.0, 4.5 and 6.0, leave the smallest — category 1 — with 822 districts, which is comfortably enough.

2.3 Does it actually matter? Measure it

The point of this course is that the previous paragraph is a hypothesis until someone counts. Comparing each split's income-category proportions against the full dataset's:

Split	Largest error in stratum proportion
purely random	up to 6.4%
stratified	up to 0.36%

Roughly eighteen times better, for one keyword argument. Note the shape of the argument: the stratification was justified by an expert's claim, and then the claim was checked with a measurement rather than deferred to.

2.4 Assert after every structural step

Three lines, three real bugs

```
assert len(strat_train_set) + len(strat_test_set) == 20640
assert set(strat_train_set.columns) == set(strat_test_set.columns)
assert strat_train_set.index.intersection(strat_test_set.index).empty
```

Nothing was dropped; the two frames have the same columns; no district is in both. Each of the three has been a real bug in a real notebook.

An assertion is a claim about your data that fails *loudly*, and in a domain that otherwise fails silently that makes it the most valuable line in the file. Write one after every split, merge, reshape and drop, and count what you dropped. The question of what was dropped is then answered by the program instead of by you, every time it runs, including the times you forget to ask.

3 Exploring — the training set, and only that

From here everything is computed on the 16,512-district training split. The other 4,128 are sealed until the final evaluation. Work on a copy, so that nothing invented while exploring — a new column, a filter, a dropped row — can reach the split frame by accident, and so that every number below is honestly labelled as a training-set number.

3.1 Look at it, and tune the plot until you can

The data is geographical, so plot latitude against longitude. It looks like California, and beyond that nothing: at full opacity the ink saturates and every district is the same black dot.

Setting `alpha=0.2` changes the picture completely — the Bay Area, Los Angeles, San Diego and the Central Valley appear, because now the density is encoded in the darkness. Same data, one keyword argument. Expect to tune plotting parameters until the pattern shows; a plot that shows nothing is usually a plot, not an absence.

Adding two more channels — radius for population, colour for price — shows that prices relate strongly to location and to population density. Red on the coast, blue inland. The model will not know what “coast” means, but it will learn longitude.

3.2 A default nobody chose, and how to catch it

The three-channel plot above is conventionally drawn with `cmap="jet"`. It ran, it produced something colourful, and it is wrong.

A colour scale is a claim: equal steps in the data look like equal steps on the screen. That claim is testable — plot each colormap’s lightness against position along it.

Colormap	Lightness range	Largest reversal	Verdict
`jet`	83.0	1.06	rises, falls, rises
`turbo`	78.9	0.87	better, still not monotone
`viridis`	75.9	0.00	every step is a step

A non-monotone lightness profile invents bands the data does not have and buries detail in the yellow. For the roughly 8% of men with a red–green deficiency, `jet` reverses rank order outright. Photocopy it and it stops being a scale at all.

Failure condition — the shape of this failure

The code ran. Nothing errored. The defect was in a default nobody chose deliberately, and it took a measurement to see. That is every failure in this course, and it is only the second lecture.

3.3 Correlations, and what they cannot see

Pearson's r against the target, on the training split:

Attribute	r with `median_house_value`
`median_income`	0.688380
`total_rooms`	0.137455
`housing_median_age`	0.102175
`households`	0.071426
`total_bedrooms`	0.054635
`population`	−0.020153
`longitude`	−0.050859
`latitude`	−0.139584

Income dominates: one bar is five times the next. That is the feature we already stratified on, on the experts' word, before we had looked — and the measurement says they were right.

These are training-set values. Compute them on all 20,640 rows and the third decimal moves, which is why a number is worth nothing without the set it was measured on attached to it.

Failure condition — two things r does not tell you

r measures *linear* correlation only. A dataset can have $r = 0$ and be completely dependent — a nonlinear relationship is enough. And $r = \pm 1$ says nothing whatever about the slope: your height in inches correlates 1.0 with your height in nanometres.

3.4 The cap, made visible — and the lines that are not there

Plot the strongest predictor against the target and the cap, \$500,001, appears as a hard horizontal line: 764 districts in our training split. There are fainter horizontal lines too, and the temptation is to name them by eye. A horizontal stripe means many districts share one exact price. That is a property of the table, not of California, so it is *countable* — and squinting at a scatter plot is not counting.

A well-known description of this dataset names three fainter lines. Counted in our own training split:

Value	Districts	Verdict
\$350,000	62	real — 21× the background
\$450,000	31	marginal
\$280,000	3	indistinguishable from background

A typical price is shared by 3 districts, so the third is not a line at all. Meanwhile the five commonest values below the cap — \$137,500, \$162,500, \$112,500, \$187,500 and \$225,000 — go unmentioned in that description, and they are the strongest stripes in the plot. Every one sits on a multiple of \$12,500, which tells you something about how the survey recorded prices and nothing about the housing market.

The habit, not the numbers

Three values were asserted; one survives. Counting took one line. You will inherit descriptions of data for the rest of your career — from papers, from colleagues, from an assistant — and the cost of checking one against the data is almost always smaller than the cost of building on it.

3.5 Attribute combinations

Raw counts per district are nearly meaningless because districts differ in size. Ratios are not.

Attribute	r with price
`median_income`	0.688
`rooms_per_house`	0.144
`total_rooms`	0.137
`bedrooms_ratio`	−0.256

`bedrooms_ratio` — bedrooms as a fraction of all rooms — is far more informative than either raw count it is built from, and its sign is interpretable: a district whose rooms are mostly bedrooms is a district of small dwellings.

A warning you will need shortly

When you create combined features, check that they are not too linearly correlated with the existing ones, and in particular avoid simple weighted sums of features already present. **Collinearity** causes real problems for some models — linear regression above all. The derivation in §4 starts to show why, and solving it — the normal equation, next lecture — finishes the argument: the matrix you have to invert becomes singular, or close enough to it that the inverse is numerically meaningless.

4 Derivation: least squares, up to convexity

You will call `LinearRegression().fit()`. What does it compute, and when does that computation fail? The second half of the question is not idle: its answer is exactly the warning that ended the exploration about engineering features as weighted sums of existing ones. This lecture derives the first half of the answer — the objective, its gradient, and why its stationary point is the minimum. Solving for the parameters is the next lecture.

4.1 The model

$$\hat{y} \;=\; \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

with $\hat{y}$ the prediction, n the number of features, x_i the i-th feature and θ_j the j-th parameter. θ_0 is the **bias** or **intercept**. Writing $\mathbf{x} = (x_0, \ldots, x_n)$ with $x_0 \equiv 1$ folds the bias into the dot product:

$$\hat{y} \;=\; h_{\boldsymbol{\theta}}(\mathbf{x}) \;=\; \boldsymbol{\theta} \cdot \mathbf{x} \;=\; \boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}.$$

4.2 What "fit" means

Training means setting $\boldsymbol{\theta}$ so the model best fits the training set, which requires a measure of how badly it fits:

$$\mathrm{MSE}(\mathbf{X}, y, h_{\boldsymbol{\theta}}) \;=\; \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^{\mathsf{T}} \mathbf{x}^{(i)} - y^{(i)} \right)^2 .$$

We minimise MSE rather than RMSE because $\sqrt{\cdot}$ is strictly increasing on $[0, \infty)$, so

$$\arg\min_{\boldsymbol{\theta}} \sqrt{f(\boldsymbol{\theta})} \;=\; \arg\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \;:$$

same minimiser, simpler algebra. This is the first instance of a pattern worth naming now, because it recurs for the rest of the course: *learning algorithms routinely optimise a different function during training than the one used to evaluate the result* — because it is easier to optimise, or because it carries terms needed only during training.

4.3 The objective, its gradient, and its curvature

Stack the instances as rows of $\mathbf{X}$ and the labels into y:

$$\mathrm{MSE}(\boldsymbol{\theta}) \;=\; \frac{1}{m} \lVert \mathbf{X}\boldsymbol{\theta} - y \rVert_2^2 .$$

We are minimising a squared Euclidean norm. That is a *geometric* statement, and it is the whole content of this section.

Differentiate, and check the second order

$$\nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\intercal}(\mathbf{X}\boldsymbol{\theta} - y)\,,$$

so at a stationary point $\hat{\boldsymbol{\theta}}$,

$$\mathbf{X}^{\intercal}\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right) \;=\; \mathbf{0}.$$

Is it a minimum? The Hessian is $\nabla^2_{\boldsymbol{\theta}} \operatorname{MSE} = \frac{2}{m}\,\mathbf{X}^{\intercal}\mathbf{X}$, and for any $\mathbf{v}$,

$$\mathbf{v}^{\intercal}\mathbf{X}^{\intercal}\mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0,$$

so the Hessian is positive semi-definite and the MSE is convex. A convex function has no local minima — only a global one — so any stationary point is *the* minimum.

Convexity is not a technicality to be waved through. It is also why gradient descent, in Lecture 4, has nowhere else to stop: its steps vanish only where the gradient does, and convexity makes every such point this minimum. Whether it gets there is the learning rate's job, as Lecture 4 shows.

4.4 Where the derivation stops

We now have the stationarity condition

$$\mathbf{X}^{\intercal}\mathbf{X}\,\hat{\boldsymbol{\theta}} \;=\; \mathbf{X}^{\intercal}y$$

and know that any solution of it is *the* minimum of the MSE. Three questions are left: when it has exactly one solution, what that solution is in closed form, and what happens to it when one feature is a weighted sum of others. All three are answered by solving for $\hat{\boldsymbol{\theta}}$ — the **normal equation**, where the next lecture starts.

Checked on our own data

The notebook checks both halves of this section on the training split before anything is fitted. The gradient formula agrees with a central finite difference of the MSE to rounding, and the Hessian $\frac{2}{m}\mathbf{X}^{\intercal}\mathbf{X}$ — eight standardised features and a column of ones — has no negative eigenvalue.

5 The notebook

The Lecture 2 notebook runs on Colab's free CPU tier and loads the data itself, so it stands on its own. It contains the stratified split and the sampling-bias measurement; the cap and the three asserted lines, counted on the training split; the map, the colour-scale test, the correlations and the three ratios; and the two numerical checks of §4 — the gradient against finite differences, and the Hessian's eigenvalues.

What to change

Run it top to bottom once. Then change one thing — the number of income bands in `pd.cut` — and watch the sampling-bias table. Two lines, and it is the whole argument of §2: too few strata and the stratification buys you little; too many and each stratum is too small for its own proportion to be estimated reliably, so the cure starts to resemble the disease.

6 Design exercise: a rental marketplace's suggested listing price

The lecture closes with a design exercise in the manner of an ML system-design interview, on a task with the shape of today's data and today's model.

The task. A rental marketplace — the app where landlords post flats and tenants find them — wants its posting form to suggest a monthly rent as the landlord types in the flat, and to badge the listing "fairly priced" for tenants. The app earns a fee per flat let through it, so it measures how fast listings let and how many landlords post: a suggestion too high leaves the flat sitting while the landlord tries a rival, one too low costs the landlord money and the app its trust. Given: the app's listings — area, rooms, floor, year built, furnished, neighbourhood, asking rent, date posted — and the lease rent of the flats let through it. A few million listings in many cities, tens of thousands new a month, neighbourhoods with thousands of listings and some with a few dozen. The output feeds the posting form, the badge tenants see and the landlord's dashboard. The model at the centre is a linear model fitted by least squares, at the end of a pipeline that was split before anyone looked.

1. **Frame it.** Supervised regression, batch: one number per flat, a label on every flat that was let, a history that fits in memory. Least squares minimises the MSE and the RMSE is reported, in euros a month, because it is in the units of the rent and has the same minimiser. The product team's criterion is relative: within about a tenth of what the flat lets for, or landlords ignore it. *Pitfall:* an RMSE in euros is dominated by the few penthouses, where a mistake of hundreds is a few per cent, while the app's volume is the cheap flats, where the same mistake is a quarter of the rent and the flat sits; the headline can improve while time-to-let, the business's number, gets worse. Write the metric beside the criterion before anything is fitted, and report the error as a share of the rent, by rent band, next to the RMSE.

2. **The label.** One row per flat that was let, the label the rent in the lease, the date a column because rents drift. The asking rent on every other listing is what a landlord hopes for, not a label. `head()`, `info()`, `describe()` and histograms on the whole history, once. *Pitfall:* every listing has an asking rent and only a fraction a lease, so the first attempt trains on asking rents, including the flats that never let because they asked too much; the suggestion then repeats the prices that failed, and time-to-let worsens on the app's own advice. Label with let flats only, keep the asking rent as a feature at most, train on recent leases, and state which months the number describes.

3. **Split before you look, and measure the split.** Set the test flats aside before one variable is plotted against another; stratify on what the product team says drives rent, floor-area bands, and measure the stratum error of a random split against the stratified one; assert that nothing was dropped, that both sides have the same columns, and that no listing and no re-listed flat

is on both. *Pitfall, data snooping:* the map of rents, the correlations and the ratios are drawn on the whole history, the features chosen from them, and only then is a test set cut; nothing errors, and the test error the release rests on is optimistic by an amount the code cannot tell you. Split first, explore a copy of the training flats, and label every exploration number a training-set number.

4. **Features, inside a pipeline.** Ratios where landlords think in ratios — rent per square metre, area per room — because a ratio of two weak columns can beat both. The neighbourhood's typical rent per square metre is the strongest single feature, and it is learned from the labels. Every learned step — the median that fills a missing floor, the encoding of the neighbourhood, that typical rent — is a transformer inside one `Pipeline`, fitted on that fit's training flats only. *Pitfall, leakage through a neighbourhood average:* the average rent per neighbourhood is computed once over the whole history and joined on as a column, so every test flat carries a feature that contains its own rent and the test error falls for a reason that will not be there when a landlord posts a new flat; the suggestion ships worse than reported. Compute the average inside the pipeline from that fit's training flats, and give a neighbourhood with a few dozen listings the city's figure.

5. **The model.** $\hat{y} = \boldsymbol{\theta}^{\top}\mathbf{x}$, fitted by minimising the squared norm of $\mathbf{X}\boldsymbol{\theta} - y$, a convex cost whose stationary point is *the* minimum. Scaling changes nothing for least squares. A coefficient per column — euros per square metre, euros for a balcony — is the breakdown the form shows the landlord. *Pitfall, collinearity:* the form holds the floor area and a "living area" that is a fixed share of it, and an engineer adds their weighted sum as a size score; the columns are then linearly dependent, the stationarity condition has infinitely many solutions, and the solver returns one whose coefficients mean nothing while the predictions look fine, so the landlord is shown nonsense as the breakdown of their price. Build ratios, never weighted sums of existing columns, and check the correlations among the features.

6. **Cross-validation, with the fold scores reported.** Split the training flats into k folds and fit the whole pipeline k times, each time scoring the fold it did not see: k RMSEs, their mean and their spread. Every choice is made on these folds, never on the test flats, and a difference smaller than the fold-to-fold spread is not a difference. *Pitfall:* two feature sets are compared by the mean alone and the one a few euros better goes into the release, when the folds spread by tens of euros and the order flips from fold to fold. Report the fold scores beside the mean, and choose the simpler candidate when the spread swallows the gap.

7. **The test set, once.** Refit the chosen pipeline on all the training flats, `predict` on the test flats, and read one RMSE and one error as a share of rent by band; that number, with the fold spread beside it, is what the release note states. *Pitfall:* the test number disappoints, a ratio is added, the test is re-run, and the second number is no longer an estimate of anything, because the test flats have now chosen the model. The finding is the number; go back to the folds, and cut a fresh test set from new listings before the next attempt is scored.

8. **Release it, and what happens next quarter.** Hand over the fitted pipeline as one object — medians, neighbourhood figures and coefficients together — with the test error by rent band against the criterion, and the loading, split and assertions for the next refit. Roll out to a share of landlords first and read time-to-let against the landlords who did not see the suggestion: an A/B test. Refit on a schedule, on the leases signed since, measuring on them first, with the data team on call for the error by city. *Pitfall:* landlords accept the suggestion,

so next quarter's asking rents carry the model's own output and a refit on them learns the suggestion, not the market — and a form that nudges every landlord in a street to one number is a question the competition regulator asks. Refit on signed leases, never asking rents; keep the landlords without the suggestion as the control; show a band, not one figure.

9. **Ship it.** Listings → let flats, the lease rent as the label, the date kept → stratified split, measured and asserted → exploration on a copy of the training flats → ratios, the neighbourhood figure and the imputer inside one pipeline → least squares → k folds, scores reported → the test set once → A/B rollout → refit on signed leases.

7 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 3's deck and in the exercise book.

1. A colleague reports their model's error on the California housing test set, and says they chose the model by trying several on that same test set and keeping the one with the lowest error. State, in one sentence, what that number is an estimate of — and what it is *not* an estimate of. [4]

2. You are given a stratified split on income category and a naive random split of the same data. A model scored on each test set gives a similar error. Does that mean the stratification was unnecessary? Answer yes or no, with a reason. [4]

3. On the training set, `longitude` has a correlation of $r = -0.05$ with the median house value. A colleague concludes that location does not affect price, and drops the column. State the flaw in one sentence, and name the plot from this lecture that refutes it. [3]

4. The course reports the RMSE of a regression model, yet training minimises the MSE. Explain why the two give the same fitted parameters, and say why the RMSE is the one reported. [3]

5. The Hessian of the MSE of a linear model is $\frac{2}{m}\mathbf{X}^\intercal\mathbf{X}$. Show that it is positive semi-definite, and state what that guarantees about a point where the gradient vanishes. [5]

Summary

The split came *before* the exploration, stratified on the predictor the experts named — and measured at 0.36% worst-case stratum error against 6.4% for a random split. Only then the exploration, on the training set alone: income dominates the correlations at 0.688, a default colormap turned out to be indefensible under a one-line test, three asserted survey artefacts reduced to one under counting, and a ratio of two weak columns beat both.

Then the first half of least squares. The MSE of a linear model is $\frac{1}{m}\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2$, minimised in place of the RMSE because the square root preserves the minimiser; its gradient is $\frac{2}{m}\mathbf{X}^\intercal(\mathbf{X}\boldsymbol{\theta} - y)$; and its Hessian $\frac{2}{m}\mathbf{X}^\intercal\mathbf{X}$ is positive semi-definite, so the MSE is convex and any stationary point is the global minimum. Solving that stationarity condition is the normal equation, and the next lecture.

The closing design exercise built the suggested listing price behind a rental marketplace's posting form in nine steps — a metric written beside a relative criterion, let flats as the label, a stratified split before the exploration, a neighbourhood figure computed inside the pipeline, ratios instead of weighted sums, fold scores reported, the test set used once, and a refit on signed leases behind an A/B rollout — each step named for the pitfall it avoids.